



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Garozashvili
Nom d'usage ▶
Prénom ▶ Konstantine
Adresse ▶ 73 Chemin de Saint-Henri 13016 , Marseille

Titre professionnel visé

Concepteur développeur d'applications

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



Sommaire

Exemples de pratique professionnelle

Activité-type n°1 — Développer une application sécurisée

p. 5

- Exemple n°1 — Mise en place de l'environnement de travail outillé et développement du tableau Kanban de l'application Ensemble (interfaces utilisateur et composants métier)

p. 5

- Exemple n°2 — Développement du site vitrine et de sa console d'administration sécurisée (liste d'attente, authentification par jeton signé)

p. 8

Activité-type n°2 — Concevoir et développer une application sécurisée organisée en couches

p. 11

- Exemple n°1 — Conception de la base de données SaaS (MCD → MLD → PostgreSQL) et développement des composants d'accès aux données SQL et NoSQL

p. 11

- Exemple n°2 — Conception de l'architecture hexagonale du desktop et de l'orchestration multi-fournisseurs des agents IA (plan structuré, critères d'acceptation, portes qualité)

p. 13

Activité-type n°3 — Préparer le déploiement d'une application sécurisée

p. 18

- Exemple n°1 — Préparation et exécution des plans de tests des deux applications (unitaires, intégration, vérification post-build, jeu d'essai)

p. 16

- Exemple n°2 — Automatisation de la release desktop et du déploiement continu du site sur VPS dans une démarche DevOps

p. 18

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. 21

Déclaration sur l'honneur

p. 22

Documents illustrant la pratique professionnelle *(facultatif)*

p. /

Annexes

p. 24

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer une application sécurisée

Exemple n°1

Mise en place de l'environnement de travail outillé et développement du tableau Kanban de l'application Ensemble (interfaces utilisateur et composants métier)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Ensemble (dépôt kanban-agent) est une application de bureau macOS que j'ai conçue et que je développe seul dans le cadre de ma formation : elle affiche les tickets GitHub d'un projet sous forme de tableau Kanban et permet de confier un ticket à un agent IA qui implémente la demande de bout en bout (branche, code, tests, pull request). Cet exemple couvre la mise en place de l'environnement de travail puis le développement du cœur visible de l'application : le tableau Kanban et ses composants métier.

J'ai d'abord installé et configuré mon environnement en fonction des contraintes du projet : macOS avec la chaîne Rust (rustup, cargo, clippy, rustfmt) pour le backend Tauri, Node.js et npm pour le frontend React/TypeScript, et Docker pour les services annexes. J'ai créé le dépôt GitHub, puis j'ai verrouillé la qualité par l'outillage avant d'écrire la première fonctionnalité : deux rulesets GitHub (protect-main interdit le push direct sur main et exige une CI verte avant fusion ; pre-prod production encadre les tags de release), des hooks de pre-commit (husky + lint-staged : ESLint corrige les fichiers TypeScript stagés, cargo fmt formate le Rust stagé, commitlint valide le format Conventional Commits), et trois workflows GitHub Actions bloquants (vérifications Rust, vérifications frontend, audit de sécurité). Même moi, propriétaire du dépôt, je ne peux pas fusionner une PR rouge — je me suis volontairement retiré ce pouvoir.

J'ai ensuite développé le tableau Kanban. Chaque carte de ticket est un composant React mémoisé qui ne s'abonne qu'à l'état de son ticket via des sélecteurs Zustand fins : ma première version, avec un seul gros abonnement au store, faisait clignoter tout le tableau dès qu'un agent streamait ses logs — j'ai profilé avec React DevTools, constaté les re-rendus en cascade, et corrigé. Le déplacement d'une carte entre colonnes est une mise à jour optimiste : l'interface bouge immédiatement, la mutation GraphQL part en arrière-plan vers GitHub Projects v2, et en cas d'échec je restaure la carte exactement où elle était. J'ai dû résoudre un vrai problème de concurrence : quand on déplace deux fois la même carte rapidement, le premier appel peut échouer après que le second a réussi, et son rollback « annulait » un déplacement valide — j'ai ajouté un compteur de génération par ticket pour que seul le dernier mouvement ait le droit de faire un rollback.

Côté composants métier, j'ai développé en Rust pur la génération des noms de branche : un type BranchName qui ne peut être construit que par le chemin validé, avec une slugification Unicode (normalisation NFKD, suppression des diacritiques : « Créer l'écran » → feat/42-creer-l-ecran). J'ai aussi internationalisé toute l'interface (anglais/français, plus que 1 762 clés de traduction avec un test automatique qui vérifie la parité des deux dictionnaires) et appliqué les bases du RGAA : navigation clavier, contrastes vérifiés sur les deux thèmes, labels explicites.

Ce travail s'est étalé sur plusieurs itérations d'une semaine environ, du printemps 2026 à juillet 2026, gérées dans GitHub Projects — avec mon propre outil comme client quotidien de son propre tableau.

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

Matériel : MacBook Air Apple Silicon. (ps. après des années sous Windows, un vrai changement de vie.)

Environnement : macOS, VS Code/Cursor, Git, GitHub (Issues, Projects v2, Actions, rulesets). Backend : Rust stable, Tauri 2, octocrab pour l'API REST GitHub et des requêtes GraphQL maison pour Projects v2, thiserror pour les erreurs typées (AppError), keyring pour le trousseau macOS. Frontend : React 19, TypeScript strict (aucun any), Vite, Zustand + immer pour l'état (cinq stores découplés), @dnd-kit pour le glisser-déposer, Vitest et React Testing Library pour les tests. Qualité : ESLint zéro warning, cargo clippy -- -D warnings (tout warning est une erreur), cargo fmt --check, husky + lint-staged + commitlint en pre-commit, CI GitHub Actions bloquante. Documentation en anglais dans le dépôt (README, docs d'architecture), conventions de commit Conventional Commits.

3. Avec qui avez-vous travaillé ?

C'est un projet personnel mené en autonomie complète : j'assume seul les rôles de développeur front, développeur back, architecte et chef de projet. J'ai échangé régulièrement avec mon référent pédagogique de La Plateforme pour valider la couverture du référentiel par mes choix techniques. Pour les points durs (modèle de données GraphQL de Projects v2, subtilités de Tauri), je me suis appuyé sur les documentations officielles en anglais. Enfin, je suis mon propre utilisateur quotidien : je gère le développement d'Ensemble... avec Ensemble, ce qui me donne un retour d'usage permanent.

4. Contexte

Nom de l'entreprise, organisme ou association - *La Plateforme*

Chantier, atelier, service - **Projet personnel de fin de formation, présenté pour la session 2026 du Titre Professionnel Concepteur Développeur d'Applications (RNCP 37873).**

Période d'exercice - début du Octobre 2025 au 20 Novembre 2025 (itérations « environnement », « board Kanban », « déplacement de tickets »).

5. Informations complémentaires *(facultatif)*

Ce projet est né d'un besoin personnel très concret : en alternance, je travaillais sur deux projets distincts, j'avais mes exercices d'école et je rédigeais mon dossier CDA — je devais constamment jongler entre plusieurs contextes techniques. La sortie du Cursor SDK a été le déclencheur — j'ai compris que l'orchestration d'agents IA directement sur le poste de travail ouvrait des possibilités énormes pour automatiser le travail répétitif et me laisser plus de concentration sur l'essentiel.

Côté technique, Rust n'était pas un choix gratuit. J'avais déjà effleuré le langage pour développer une petite application de surveillance des températures de mon portable, qui m'avait permis de comprendre l'architecture d'une app de bureau et de me familiariser avec Tauri. En reprenant Rust pour Ensemble, j'ai été convaincu par son modèle de propriété : pas de ramasse-miettes imprévisible, une consommation mémoire maîtrisée et une sécurité thread native qui sont déterminantes quand on exécute des sidecars d'agents, des appels réseau et une interface réactive dans la même application. Rust me demande plus de rigueur, mais il me donne en retour des programmes prévisibles, rapides et solides — exactement ce que je voulais pour un outil que j'utilise moi-même tous les jours.

Le dépôt kanban-agent compte à ce jour 563 pull requests fusionnées, toutes passées par le flux branche → PR → CI verte → fusion : la démarche qualité décrite ici n'est pas théorique, elle est visible dans l'historique Git. Les extraits de code significatifs (carte mémorisée, déplacement optimiste avec compteur de génération, BranchName et sa slugification Unicode) figurent au chapitre 6 de mon dossier de projet, avec les tests unitaires associés.

Activité-type 1 Développer une application sécurisée

Exemple n° 2 - Développement du site vitrine et de sa console d'administration sécurisée (liste d'attente, authentification par jeton signé)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le second projet de mon écosystème, ensemble-website (en production sur konstantine.fr), est le site public du produit : vitrine, inscription à une liste d'attente, distribution des mises à jour de l'application desktop. Cet exemple couvre deux réalisations de début juillet 2026 : la page publique d'inscription à la liste d'attente, et la console d'administration cachée qui permet de suivre les inscriptions — avec son authentification.

Pour la page waitlist, j'ai conçu une interface qui raconte le produit : le formulaire d'inscription est mis en scène comme un ticket Kanban qui, à la soumission, glisse de la colonne « Backlog » vers la colonne « Queued » (animation Framer Motion, avec une version statique pour prefers-reduced-motion). Côté serveur, l'inscription est un upsert idempotent : l'email est normalisé (minuscules, trim), validé par DTO (@IsEmail, pipe global en mode whitelist qui supprime tout champ non déclaré — anti mass-assignment), et l'unicité est garantie par un index unique PostgreSQL — cliquer deux fois sur « S'inscrire » ne produit jamais d'erreur.

Pour la console d'administration, la contrainte que je me suis fixée était : une route cachée (/admin, non référencée — noindex, aucun lien public), mais dont la vraie sécurité est côté API, jamais côté interface. J'ai développé un module NestJS complet : POST /api/admin/login vérifie les identifiants configurés en variables d'environnement avec une comparaison en temps constant (crypto.timingSafeEqual, pour ne pas divulguer d'information par le temps de réponse) et un anti-force-brute (5 échecs maximum par fenêtre de 15 minutes et par IP). En cas de succès, il émet un jeton de session signé HMAC-SHA256 de structure base64url(payload).base64url(signature) — le même principe qu'un JWT : le payload porte l'identité et une expiration (12 heures), la signature garantit l'intégrité, et le serveur ne stocke aucune session (jeton stateless : un redémarrage ne déconnecte personne, et la révocation globale se fait en changeant le secret de signature). Un guard NestJS vérifie ce jeton Bearer sur chaque endpoint de données (/api/admin/stats, /api/admin/waitlist) ; sans jeton valide, la réponse est 401. Le tableau de bord affiche les indicateurs (inscriptions du jour/de la semaine, comptes, abonnements, téléchargements) et un graphique des inscriptions sur 30 jours.

J'ai vérifié le comportement de bout en bout avant de livrer : mauvais mot de passe → 401, accès aux statistiques sans jeton → 401, avec jeton → données réelles, pagination de la liste des inscrits.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Backend : NestJS 11 (modules, injection de dépendances, ValidationPipe global, guards), Prisma sur PostgreSQL 16, module crypto natif de Node (HMAC-SHA256, timingSafeEqual) — zéro dépendance ajoutée pour l'authentification. Frontend : Next.js 15 en export statique, React 19, Tailwind CSS v4, composants dans l'esprit shadcn/ui (cartes, tables, badges) construits sur les tokens du design system Ensemble, graphique SVG accessible (table de données pour lecteurs d'écran). Secrets : ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_SESSION_SECRET fournis par variables d'environnement — en production, provisionnés par les secrets chiffrés GitHub Actions à chaque déploiement, jamais commités. Outils : Docker Compose pour la base locale, Prisma Studio, curl pour les tests d'API, pnpm en workspace.

3. Avec qui avez-vous travaillé ?

En autonomie complète sur le code. J'ai joué moi-même le rôle du « client » en rédigeant l'expression de besoin (une console pour suivre la traction du produit sans ouvrir la base en SSH), et le rôle de l'attaquant en testant les cas d'abus (force brute, jeton falsifié, champ inattendu dans le payload). Les choix cryptographiques (HMAC-SHA256, comparaison temps constant, expiration courte) s'appuient sur les recommandations OWASP (Authentication Cheat Sheet) et sur la documentation Node.js officielle.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme

Chantier, atelier, service ▶ Projet personnel de fin de formation site public konstantine.fr du produit Ensemble.

Période d'exercice ▶ du 5 juin 2026 au 12 juin 2026 (itération « waitlist + console d'administration »).

5. Informations complémentaires *(facultatif)*

Le détail du format de jeton et la comparaison avec un JWT standard figurent au chapitre 7 (sécurité) de mon dossier de projet. La console est en production ; ses endpoints sont observables (401 sans jeton) mais la route n'apparaît ni dans la navigation ni dans les sitemaps. J'ai choisi volontairement de signer un jeton maison plutôt que d'installer une librairie JWT : pour un seul rôle administrateur, 130 lignes lisibles et testables m'ont paru préférables à une dépendance de plus — je sais expliquer chaque octet du jeton émis.

Activité-type 2

Exemple n° 1 -

Concevoir et développer une application sécurisée organisée en couches
Conception de la base de données SaaS (MCD → MLD → PostgreSQL) et
développement des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le site Ensemble devait passer d'une simple liste d'attente à un vrai modèle SaaS. La particularité métier qui a guidé toute la conception : l'application exécute les modèles d'IA en local, sur la machine de l'utilisateur — le serveur ne vend donc pas du calcul, il vend des droits d'usage (le modèle JetBrains/Raycast) : compte, abonnement, clé de licence, nombre de postes autorisés, quotas mensuels.

Je suis parti des besoins fonctionnels pour recenser les entités et construire le MCD (9 entités : utilisateur, plan, abonnement, licence, activation, compteur d'usage, release, téléchargement, inscription waitlist), que j'ai dérivé en MLD puis en modèle physique PostgreSQL (10 tables au moment de la migration `saas_entitlements`, 12 tables aujourd'hui avec l'ajout ultérieur des messages de contact et de la configuration SMTP). Les règles de gestion structurantes : un plan est un tarif sous forme de données (quota mensuel, droits aux fonctionnalités, nombre de postes — changer une offre est un UPDATE, jamais un redéploiement) ; le couple (licence, empreinte machine) est unique, c'est cette contrainte qui fait respecter le nombre de postes ; un compteur d'usage existe par (licence, fonctionnalité, mois) — contrainte `UNIQUE(licenseId, feature, period)`. J'ai fait le choix des UUID en clés primaires (rien d'inférable sur les volumes) et du mapping `snake_case` SQL / `PascalCase` code via Prisma.

Chaque évolution du schéma est une migration SQL versionnée dans Git (`0001_init`, `add_user_table`, `saas_entitlements`, `seed_plans`), appliquée en production par `prisma migrate deploy` à chaque déploiement — je n'ai jamais tapé de SQL à la main sur le serveur.

Le composant d'accès aux données dont je suis le plus fier est le contrôle de quota : l'application desktop déclare chaque run avant de le lancer (`POST /api/usage/consume`), et c'est PostgreSQL qui décide atomiquement, en une seule instruction `INSERT ... ON CONFLICT ... DO UPDATE ... WHERE used < quota` : si la garde échoue, aucune ligne n'est retournée et le serveur répond 402 Payment Required avec l'URL d'upgrade. Deux requêtes concurrentes « lire puis incrémenter » auraient pu dépasser le quota (race condition classique) ; ici la base a le dernier mot. Côté NoSQL, j'ai développé le RedisService (`ioredis`, connexion paresseuse pour survivre aux démarrages désordonnés de Docker et PM2) qui porte le rate-limiting de l'endpoint de mise à jour (60 requêtes/minute/IP, compteur Redis avec expiration) et le healthcheck `/api/health/ready` qui vérifie réellement PostgreSQL et Redis.

J'ai constitué un jeu d'essai seedé et idempotent : les 3 plans (Free 0 € / 15 runs / 1 poste ; Pro 19 € / illimité / 2 postes ; Enterprise sur devis), une utilisatrice free à 13/15 runs consommés, un utilisateur pro avec deux postes activés et son abonnement Stripe, deux releases, un téléchargement, un événement webhook traité — de quoi démontrer chaque règle de gestion en conditions réelles.

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

PostgreSQL 16 et Redis 7 en conteneurs Docker Compose (la même définition d'infrastructure en local et en production), Prisma 6 (schéma déclaratif, migrations versionnées, client typé, Prisma Studio pour l'inspection), ioredis, NestJS pour la couche services/contrôleurs, class-validator pour les DTO. Mermaid pour les diagrammes MCD/MLD (sources versionnées, rendus PNG). Tests d'intégration Vitest exécutés contre le vrai PostgreSQL local avec données jetables. pg_dump/pg_restore via le conteneur pour la sauvegarde/restauration documentée.

3. Avec qui avez-vous travaillé ?

En autonomie. La modélisation a été challengée par mon référent pédagogique (notamment le choix « le quota vit dans la table plans, pas dans le code »). Pour le patron d'incrément conditionnel atomique, je me suis appuyé sur la documentation PostgreSQL officielle (INSERT ... ON CONFLICT) et les recommandations Prisma sur \$queryRaw paramétré.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *La Plateforme*

Chantier, atelier, service ▶ *Projet personnel de fin de formation — backend SaaS du site konstantine.fr*

Période d'exercice ▶ *début juin – du 24 juin (migrations saas_entitlements et seed_plans du 24 juin 2026).*

5. Informations complémentaires (facultatif)

Le MCD et le MLD complets figurent au chapitre 4 de mon dossier de projet (diagrammes 4.8 et 4.9), avec le script SQL des migrations et la requête de contrôle du jeu d'essai. La séquence de démonstration HTTP (200 → 200 → 402 quand le quota s'épuise, 403 sur une fonctionnalité hors plan, 404 sur une clé inconnue) est reproduite au chapitre 9.

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

Exemple n° 2 - *Conception de l'architecture hexagonale du desktop et de l'orchestration multi-fournisseurs des agents IA (plan structuré, critères d'acceptation, portes qualité)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

C'est le cœur technique de l'application Ensemble : concevoir une architecture en couches où un agent IA peut analyser un ticket, produire un plan, l'implémenter, et où c'est mon code — pas l'IA — qui décide si le travail est accepté.

J'ai organisé le backend Rust en architecture hexagonale : `commands/` (handlers IPC minces), `domain/` (logique métier pure, zéro I/O, 100 % testable), et des adaptateurs (`github/` pour REST+GraphQL, `agent/` pour l'orchestration IA, `config/` pour le Keychain). Le frontend React ne parle au backend que par des wrappers typés — aucun `invoke()` dans un composant.

L'orchestration des agents fonctionne en deux phases. Analyse : l'application lance un processus Node compagnon (« sidecar ») qui pilote le CLI du fournisseur choisi (Claude, Cursor, Deepseek, Gemini/Antigravity ou Kimi — un adaptateur par fournisseur, une seule logique d'orchestration partagée). Le prompt que j'ai rédigé impose un contrat de sortie strict : un plan en prose (classification du ticket : defect/hardening/feature, cause racine, étapes, risques, stratégie de test) terminé par un bloc JSON structuré à dix clés exactement (`files_to_create`, `files_to_modify`, `files_to_delete`, `dependencies_to_add`, `test_files`, `estimated_complexity`, `client_issue_summary`, `plain_summary`, `acceptance_checks`, `ticket_class`). Ce JSON est validé côté hôte (un plan sans bloc valide est renvoyé à l'agent pour réparation), puis persisté comme artefact. Build : le plan sauvegardé est réinjecté tel quel dans le prompt de l'agent d'implémentation — qui peut être un autre fournisseur que celui de l'analyse, le JSON servant de langue commune.

Les critères d'acceptation (`acceptance_checks`) sont des commandes shell que l'agent d'analyse propose comme preuve du ticket (« ces tests passent », « ce motif a disparu du code »). Comme ils sont générés par un modèle, je les traite comme des entrées non fiables : ils traversent quatre couches de correction que j'ai développées — un lint de périmètre qui supprime les vérifications hors sujet (les portes globales lint/type-check n'ont rien à faire dans un critère de ticket), un « dry-run » qui exécute chaque critère sur la branche de base et écarte ceux qui échouent déjà avant le travail, une réécriture des filtres multiples cargo test, et une inversion des critères d'absence (un grep nu qui « vérifie que X a disparu » réussit tant que X est présent — le bug le plus vicieux du projet, un build correct restait éternellement « à corriger »). Après le build, l'hôte exécute la suite de tests du

DOSSIER PROFESSIONNEL ^(DP)

dépôt puis mes portes qualité (npm run lint, type-check, cargo fmt --check, détectées d'après les manifestes du workspace) ; le verdict final (pass | needs_fix | blocked) est calculé par une seule fonction du domaine qui combine le registre des critères et le résultat des vérifications automatiques — règle née d'un vrai bug où cocher une case manuelle pouvait repeindre en vert un build dont la suite échouait.

2. Précisez les moyens utilisés :

Rust (Tauri 2, tokio pour les processus asynchrones, serde pour les artefacts JSON), Node.js pour les sidecars (modules purs testés avec node:test/Vitest, orchestration séparée — une couture de test que je me suis imposée), CLIs des fournisseurs (claude, cursor-agent, agy, kimi) pilotés en mode headless avec sortie stream-json, GitHub Projects v2 en GraphQL. Les artefacts d'un ticket (plan markdown, plan JSON, registre d'acceptation, verdict, journaux de tests et de qualité) sont des fichiers JSON/Markdown versionnés par ticket dans le répertoire Application Support de l'application — l'UI affiche le verdict persisté, jamais une déduction refaite à partir des logs.

3. Avec qui avez-vous travaillé ?

En autonomie complète. Ce sous-système est celui où le « pair » était l'IA elle-même : j'ai passé beaucoup de temps à lire les transcriptions des agents pour comprendre leurs modes d'échec (critères inversés, plans sans JSON, sessions perdues) et à durcir mon contrat en conséquence. Les documentations de référence : Claude Code, Cursor SDK, Gemini CLI et Kimi CLI (toutes en anglais), plus les RFC/manuels POSIX pour les subtilités shell (codes de sortie de grep, pipes, EPIPE).

4. Contexte

Nom de l'entreprise, organisme ou association - La Plateforme

Chantier, atelier, service - Projet personnel de fin de formation — application desktop Ensemble (kanban-agent).

Période d'exercice - d'avril à juillet 2026 (itérations « orchestration multi-fournisseurs », « critères d'acceptation », « portes qualité »).

5. Informations complémentaires *(facultatif)*

Le chapitre 6 de mon dossier de projet documente trois incidents réels de ce sous-système et leur résolution (critère d'acceptation inversé ; crash EPIPE d'un sidecar dont le CLI ne lit jamais l'entrée standard ; commande slash invisible car préfixée par du contexte injecté), chacun couvert ensuite par un test de non-régression que j'ai vérifié en le faisant échouer contre l'ancien code.

Activité-type 3 Préparer le déploiement d'une application sécurisée

Exemple n° 1 - *Préparation et exécution des plans de tests des deux applications (unitaires, intégration, vérification post-build, jeu d'essai)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai construit le plan de tests des deux applications par couches, en visant à chaque fois le test le moins cher qui prouve le comportement.

Sur le desktop : la logique métier étant du Rust pur sans I/O, elle est couverte par des tests unitaires exhaustifs — 1 215 tests cargo test passent au moment où j'écris, incluant les cas tordus (titres de tickets 100 % emoji, plans sans bloc JSON, verdicts avec suite en échec). Les sidecars Node et le frontend suivent la même règle : 2 524 tests Vitest/node:test passent sur les modules de bibliothèque, les stores Zustand, l'i18n et les composants critiques. L'orchestration reste mince.

Sur le site : tests unitaires des services NestJS, tests des composants React (waitlist, formulaire de contact) avec Vitest et React Testing Library, et tests d'intégration contre le vrai PostgreSQL local pour le contrôle de quota (quota 3 → la 4^e demande est refusée ; plans illimités ; fonctionnalité hors plan ; limite de postes ; clé inconnue), avec des données jetables créées puis supprimées par le test. Le jeu d'essai seedé (3 plans, Marie 13/15 runs, Paul pro 2 postes) me permet de dérouler la démonstration HTTP complète : 200 → 200 → 402 quand le quota s'épuise.

Deux pratiques dont je suis particulièrement rigoureux. D'abord, chaque bug corrigé devient un test de non-régression, que je valide en le faisant échouer contre l'ancien code (git stash du correctif → le test doit être rouge → stash pop → vert) : c'est la preuve que le test garde bien la porte, qu'il n'est pas une tautologie. Ensuite, la vérification post-déploiement fait partie du plan de tests : le pipeline du site ne s'arrête pas à « deploy vert », il interroge les vrais endpoints (/api/health, /api/health/ready qui vérifie base et Redis, un aller-retour waitlist réel, la page d'accueil en 200) et échoue si l'un ne répond pas — un pipeline vert veut dire quelque chose.

Enfin, l'application desktop applique elle-même un plan de tests à chaque build d'agent : suite du dépôt cible, critères d'acceptation du ticket, portes qualité (lint, type-check, format) — décrits dans l'exemple AT2 n°2.

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

cargo test (unitaires et intégration Rust), Vitest + React Testing Library (frontend), node:test (sidecars), Supertest-équivalent via curl scripté pour les démonstrations HTTP, PostgreSQL/Redis Docker pour les tests d'intégration, seed Prisma idempotent pour le jeu d'essai, GitHub Actions pour l'exécution systématique à chaque PR (tests bloquants pour la fusion), couverture et profils React DevTools pour cibler les tests de performance de rendu.

3. Avec qui avez-vous travaillé ?

En autonomie. Mon « testeur externe », ce sont les agents IA eux-mêmes : chaque run d'agent exécute ma chaîne de vérification, ce qui soumet le plan de tests à des centaines d'exécutions réelles et fait remonter les cas limites qu'un test manuel n'aurait jamais croisés. Les incidents ainsi détectés ont systématiquement enrichi le corpus de tests.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme

Chantier, atelier, service ▶ *Projet personnel de fin de formation — les deux dépôts de l'écosystème Ensemble.*

Période d'exercice ▶ *en continu du printemps à juillet 2026.*

5. Informations complémentaires (facultatif)

Le plan de tests détaillé (types, environnements, critères d'arrêt) figure au chapitre 8 de mon dossier de projet, et le jeu d'essai au chapitre 9 avec les résultats attendus/obtenus. Les chiffres sont vérifiables dans les dépôts : les suites tournent en CI sur chaque pull request.

Activité-type 3 Préparer le déploiement d'une application sécurisée

Exemple n° 2 - Automatisation de la release desktop et du déploiement continu du site sur VPS dans une démarche DevOps

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai automatisé les deux chaînes de mise en production de l'écosystème, avec le même principe directeur : une commande, un enchaînement vérifié, un échec = tout s'arrête.

Pour le site (ensemble-website) : j'ai provisionné un VPS Hetzner (Ubuntu LTS, Nginx, PM2, Docker, UFW), rédigé la procédure d'installation reproductible (setup-vps.sh, ensure-host.sh idempotent) et le pipeline GitHub Actions déclenché automatiquement par le push d'un tag de version (git tag -a vX.Y.Z puis git push origin vX.Y.Z) : un build gate complet sur le runner (lint, type-check, tests, build front et back, validation Prisma — un seul échec et rien n'est déployé), puis le déploiement SSH (checkout du tag exact, .env régénéré depuis les secrets chiffrés, migrations avant redémarrage — si une migration échoue, l'ancienne version continue de tourner —, rebuild, PM2, reload Nginx), puis l'étape Verify qui teste le comportement réel en production.

Pour le desktop (kanban-agent) : j'ai développé un script de release une-commande (npm run release:deploy -- 2.1.11) qui enchaîne la montée de version synchronisée sur package.json, tauri.conf.json et Cargo.toml (Cargo.lock est mis à jour indirectement par la compilation Rust), le commit et le tag Git sur la branche courante, la construction du DMG signé avec ses assets de marque régénérés depuis les SVG sources, le calcul du SHA-256, la mise à jour du manifeste public latest.json dans le dépôt ensemble-website, l'upload du DMG sur le VPS, le déclenchement du déploiement du site, et une vérification finale : le script relit le manifeste en production et échoue si la version publiée n'est pas celle attendue. Les utilisateurs existants sont prévenus par l'application elle-même (vérification horaire du manifeste, toast « Update available » — un flux de mise à jour « soft », choisi et assumé car l'application n'est pas notariée Apple).

Cet exemple me permet aussi de rendre compte d'un incident de production réel et de la réponse apportée, en juillet 2026 : en pleine release 2.1.11, GitHub Actions a refusé de démarrer le moindre job — blocage de facturation au niveau du compte (« recent account payments have failed »). Le DMG était construit et uploadé, mais le site ne se redéployait plus : le manifeste public restait à l'ancienne version. Diagnostic : la dépendance à l'infrastructure d'exécution hébergée de GitHub était un point de défaillance unique que je ne contrôlais pas.

Réponse structurée : j'ai d'abord isolé la cause (le job échouait avant toute étape, sur tous les workflows, y compris un job Linux trivial — donc ni mon code ni mes workflows), puis j'ai enregistré des runners auto-hébergés sur mon VPS (services systemd, un runner par dépôt) et basculé les jobs Linux sur ces runners. Les jobs macOS restent temporairement sur les runners hébergés par GitHub en attendant la remise en ligne du runner Apple Silicon local ; l'essentiel du pipeline (lint, tests, build gate, déploiement du site) ne dépend plus de la facturation GitHub. J'ai documenté le compromis de sécurité (un runner exécute le code des workflows sur la machine de production — acceptable ici : dépôts privés, un seul contributeur).

2. Précisez les moyens utilisés :

VPS Hetzner CX33 (Ubuntu LTS), Nginx + Let's Encrypt (TLS, HSTS), UFW (22/80/443 uniquement), PM2, Docker Compose (PostgreSQL + Redis liés à 127.0.0.1), GitHub Actions (workflows YAML, secrets chiffrés, environnements), runners auto-hébergés GitHub sur le VPS (services systemd, jobs Linux basculés), clés SSH dédiées à moindre privilège (clé CI → VPS ; deploy key en lecture seule VPS → dépôt), bash avec set -euo pipefail pour tous les scripts de release, dmgbuild pour la construction du DMG (écriture directe du .DS_Store, après avoir diagnostiqué que le scripting Finder était non fiable), codesign avec identité stable, shasum pour les empreintes.

3. Avec qui avez-vous travaillé ?

En autonomie complète, avec les documentations Hetzner, GitHub Actions (hosted vs self-hosted runners, sécurité des runners), Tauri (bundling, signature) et Apple (codesign). L'incident de facturation a été l'occasion d'un vrai raisonnement d'architecte seul face à une production en panne : identifier la cause exacte avant d'agir, choisir une solution qui supprime la classe de problème (et pas seulement l'occurrence), documenter le compromis.

DOSSIER PROFESSIONNEL ^(DP)

4. Contexte

Nom de l'entreprise, organisme ou association ▶ *La Plateforme*

Chantier, atelier, service ▶ *Projet personnel de fin de formation — production konstantine.fr + distribution de l'application desktop.*

Période d'exercice ▶ *de fin juin à juillet 2026 (mise en production initiale le 30 juin 2026, releases 2.1.x en continu ensuite).*

5. Informations complémentaires *(facultatif)*

Les procédures complètes (DEPLOYMENT.md, docs/CICD.md) sont versionnées dans les dépôts pour qu'une autre personne puisse déployer sans moi. Les chapitres 5 (procédure) et 7 (sécurité de la chaîne) de mon dossier de projet détaillent chaque étape ; l'état de la production est vérifiable publiquement (<https://konstantine.fr>, manifeste /updater/latest.json, page /download avec le SHA-256 affiché).

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Titre professionnel de Technicien d'assistance en informatique	Ministère du Travail, du Plein Emploi et de l'Insertion (établi par la Direction régionale de l'économie, de l'emploi, du travail et des solidarités Provence-Alpes-Côte d'Azur)	5 mars 2024
Développeur web et web mobile	Le directeur régional de l'économie, de l'emploi, travail et des solidarités Provence-Alpes-Côte d'Azur (Ministère du Travail, de la Santé, des Solidarités et des Familles)	3 octobre 2025

Déclaration sur l'honneur

Je soussigné **Konstantine Garozashvili** déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur des réalisations présentées.

Fait à Marseille le 29/06/2026

pour faire valoir ce que de droit.

Signature :

A handwritten signature in black ink, appearing to read 'K. Garozashvili', with a large, stylized 'G'.

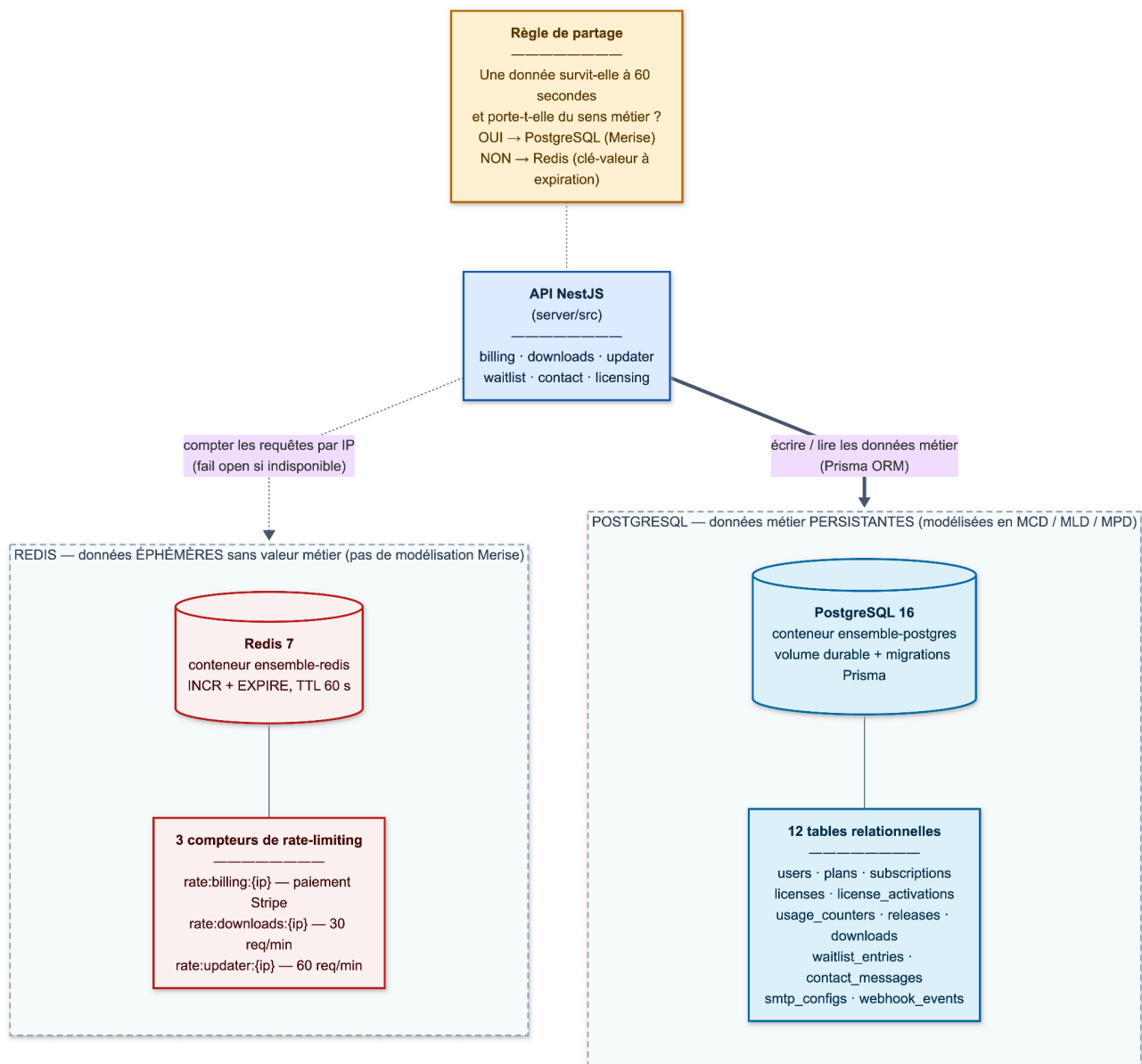
Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

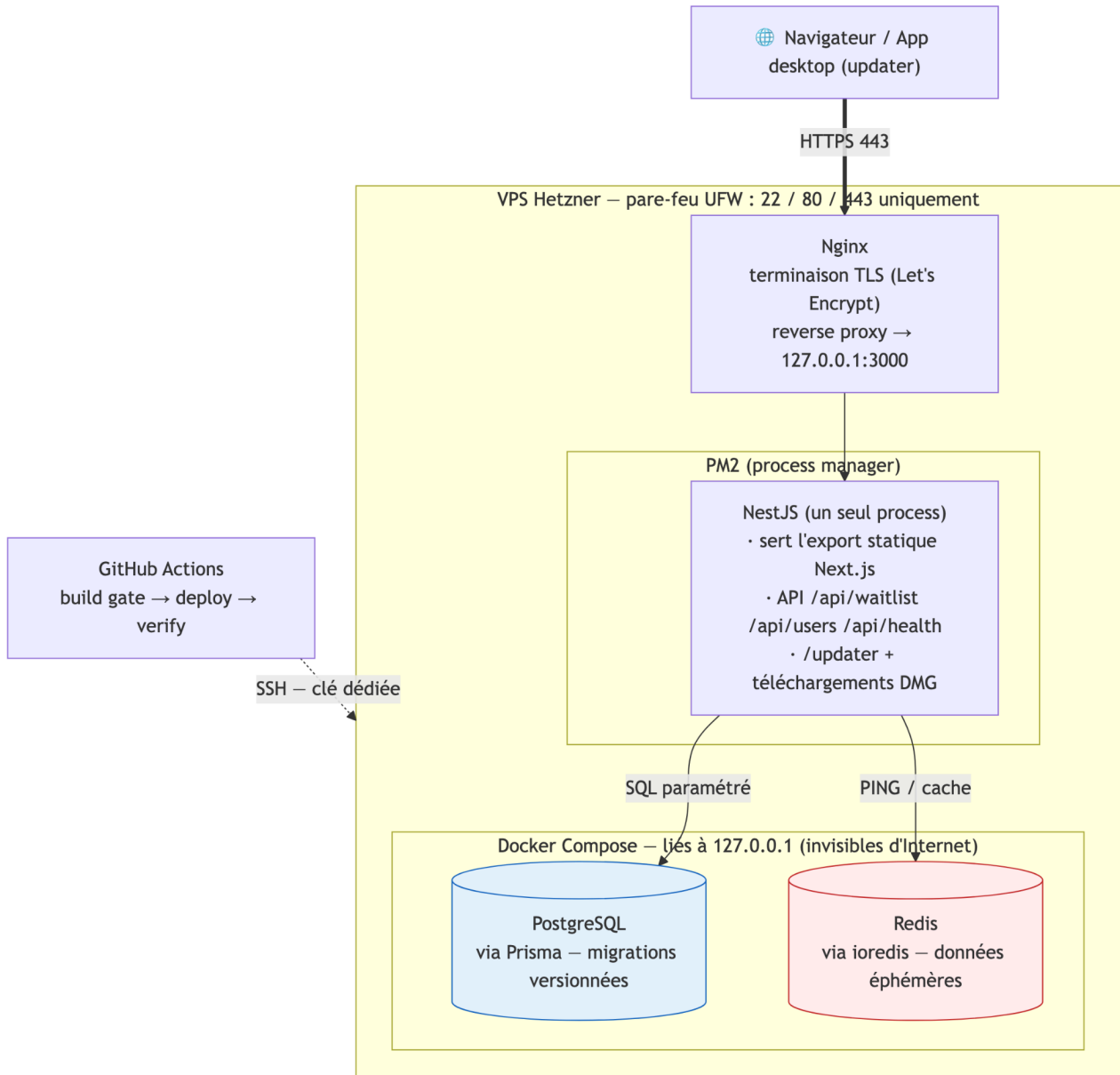
ANNEXES

ENSEMBLE WEBSITE — Architecture de stockage · PostgreSQL (persistant) vs Redis (éphémère)



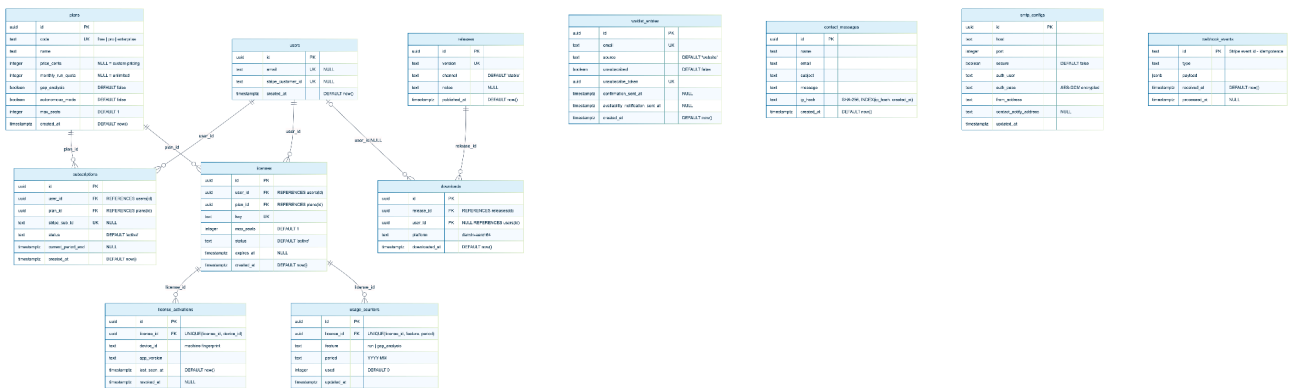
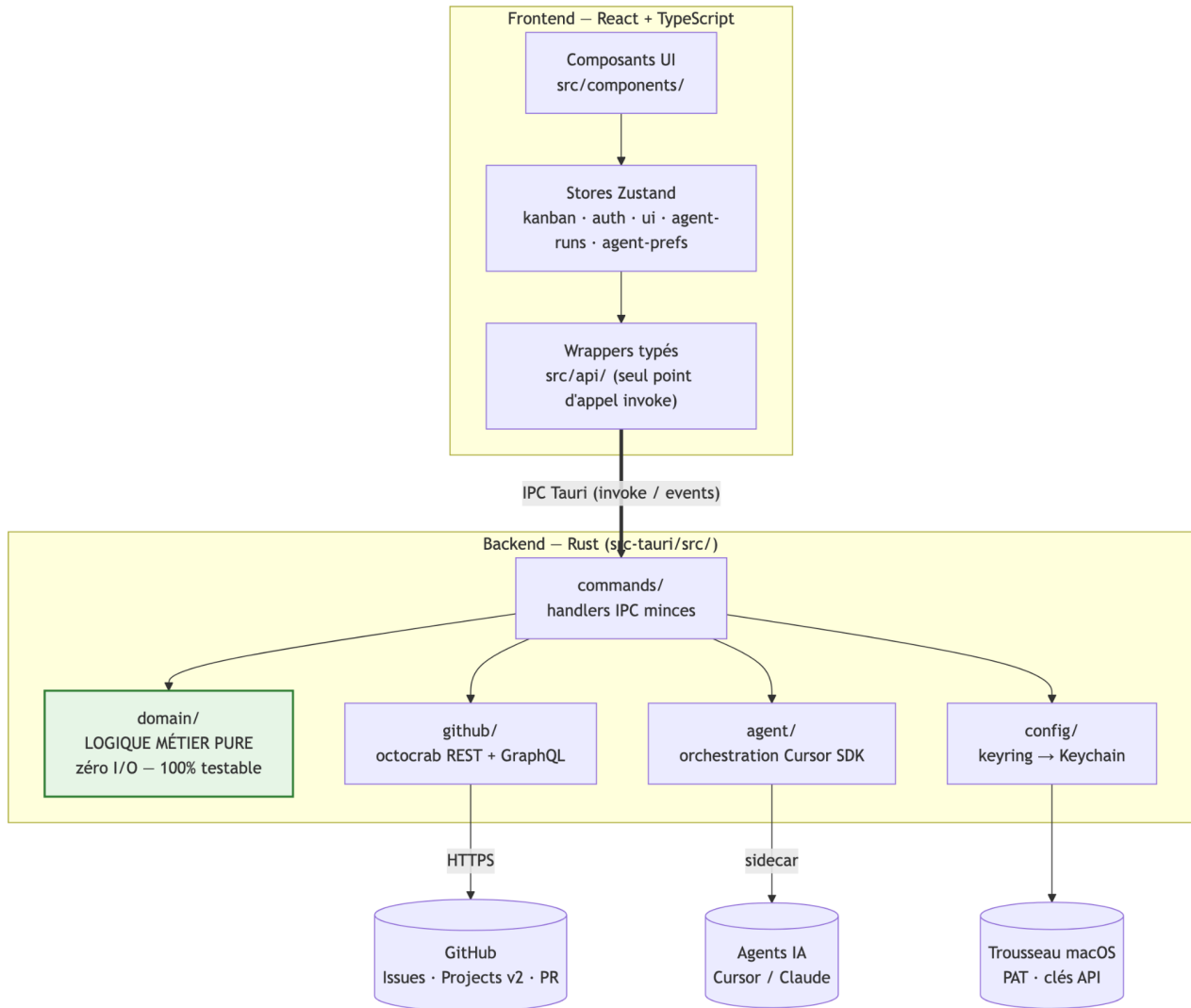
DOSSIER PROFESSIONNEL (DP)

Architecture répartie – Ensemble Website (production)



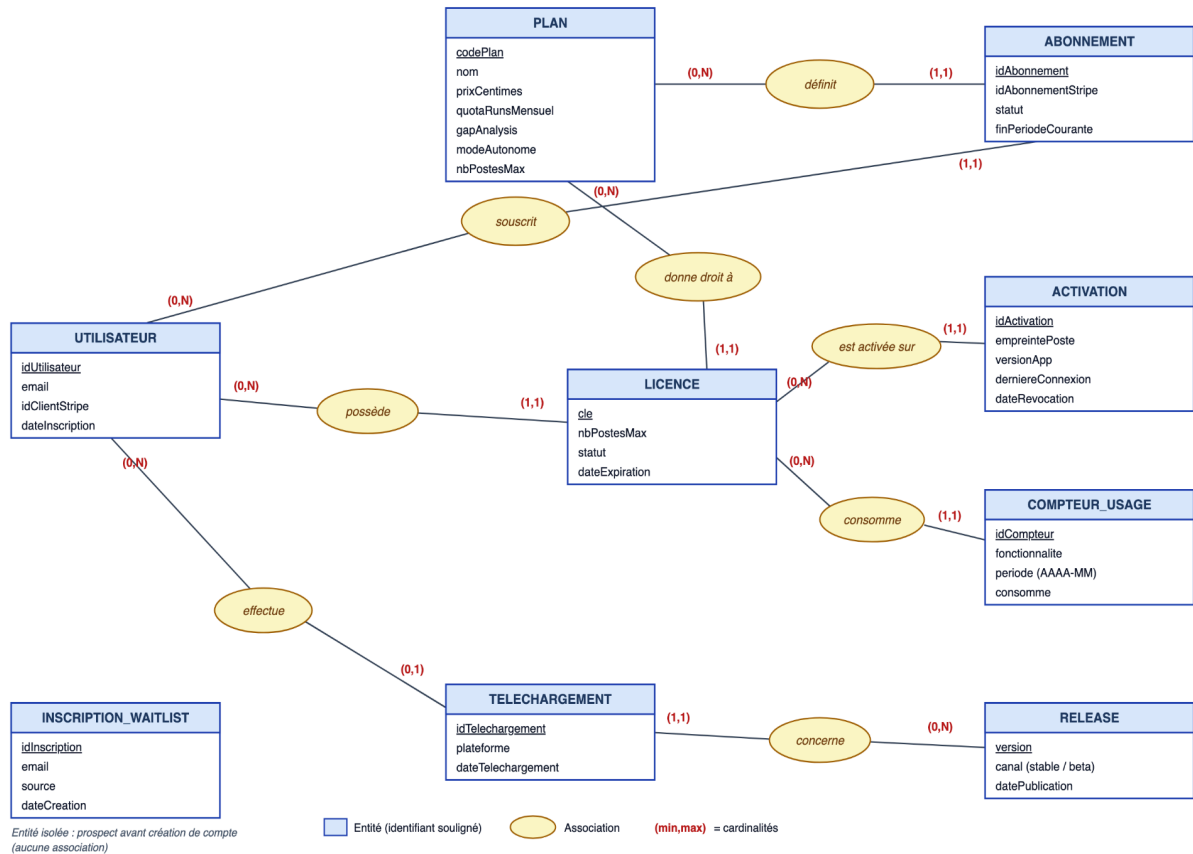
DOSSIER PROFESSIONNEL (DP)

Architecture hexagonale – Ensemble (kanban-agent)



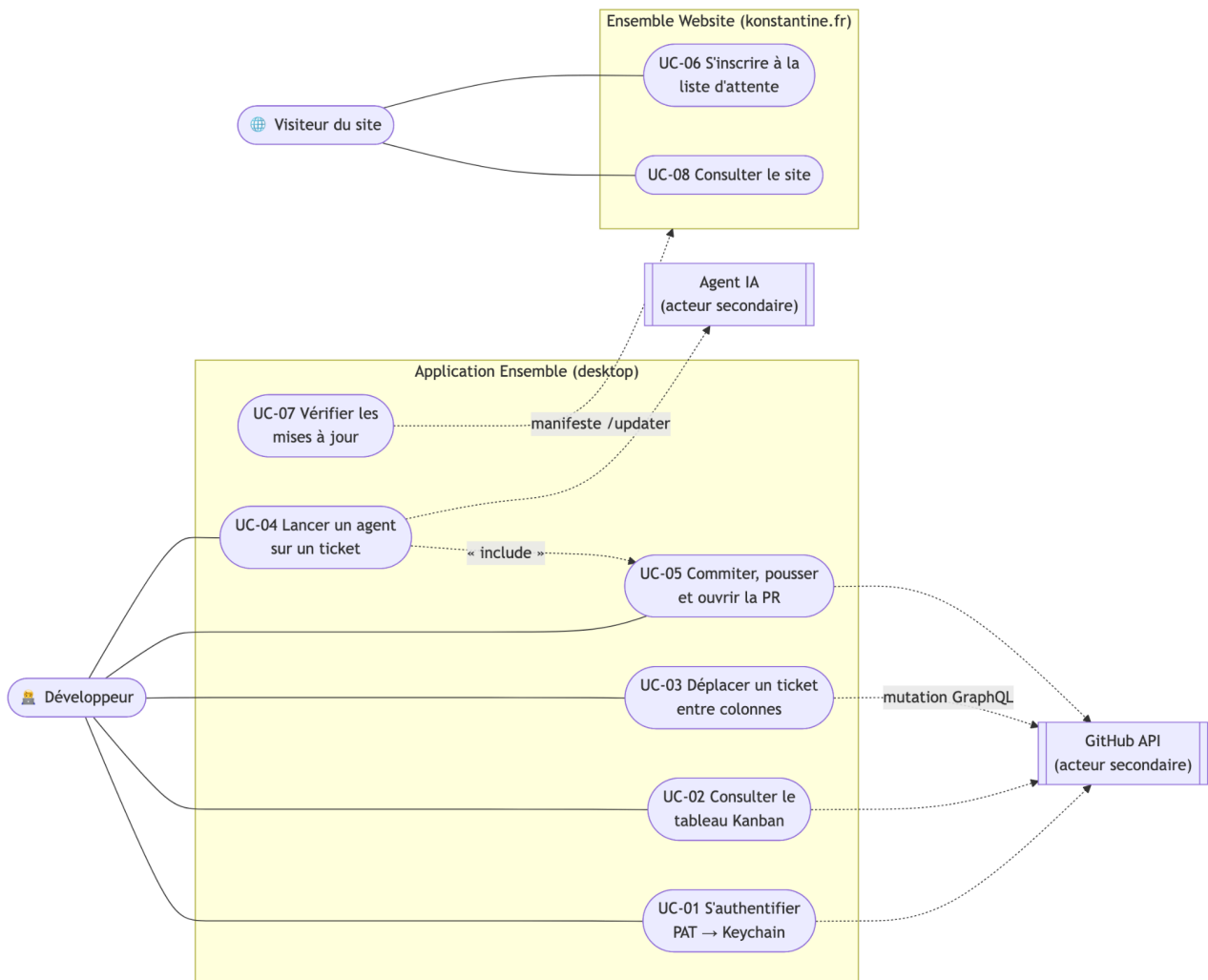
DOSSIER PROFESSIONNEL (DP)

MCD – Modèle conceptuel de données (formalisme Merise) – Ensemble Website



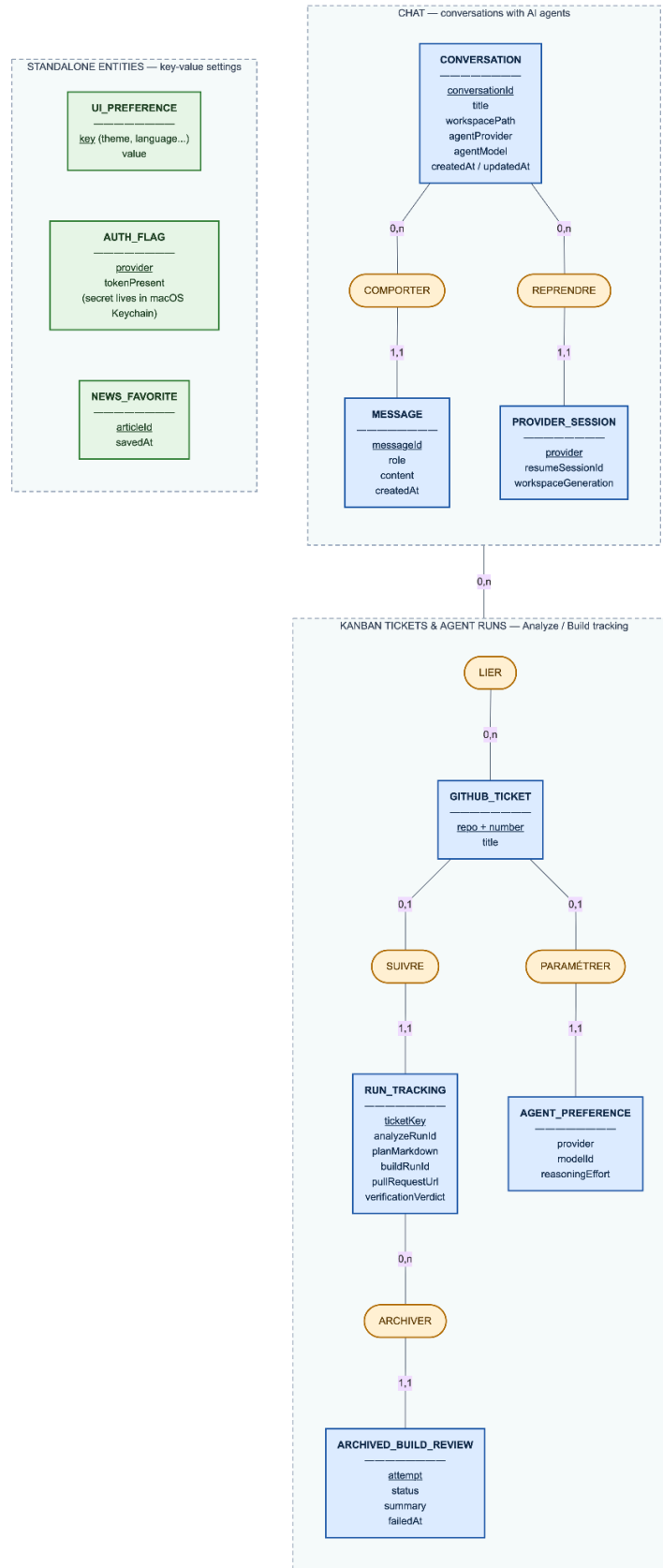
DOSSIER PROFESSIONNEL ^(DP)

Diagramme de cas d'utilisation – écosystème Ensemble

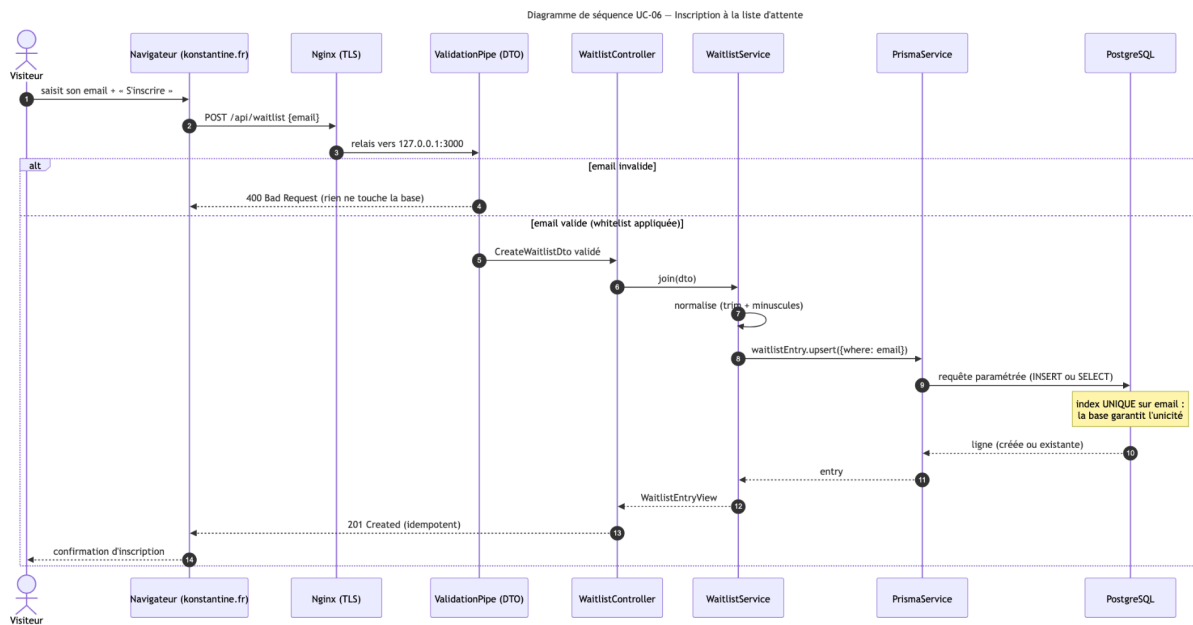
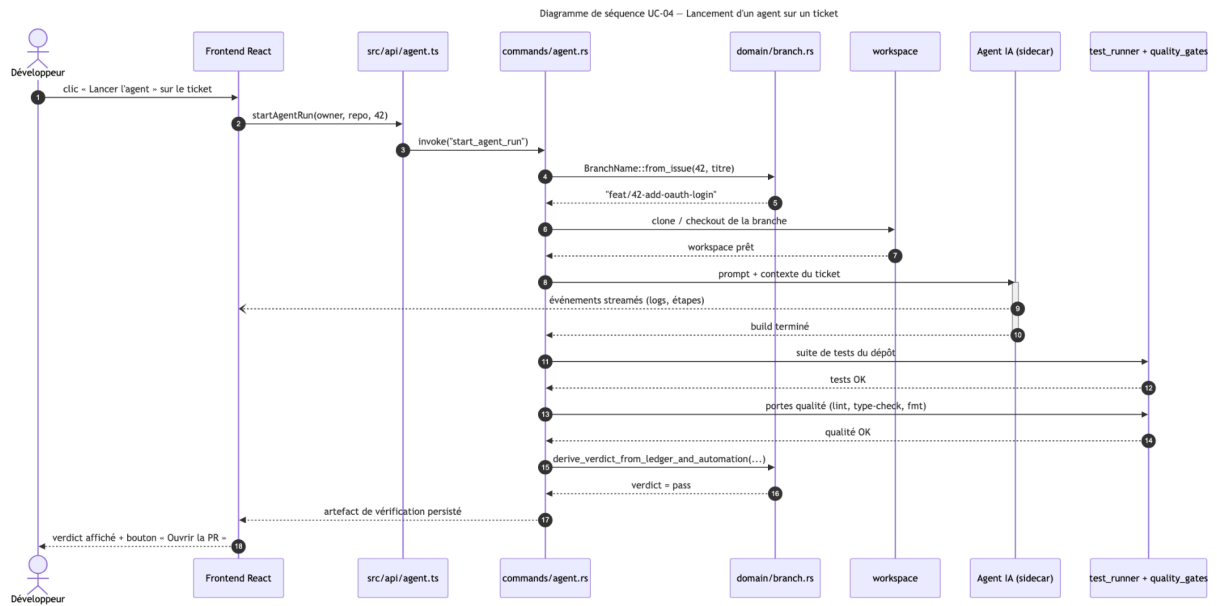


DOSSIER PROFESSIONNEL (DP)

KANBAN-AGENT — MCD · Conceptual Data Model (Merise) · local persistence (localStorage + macOS Keychain)

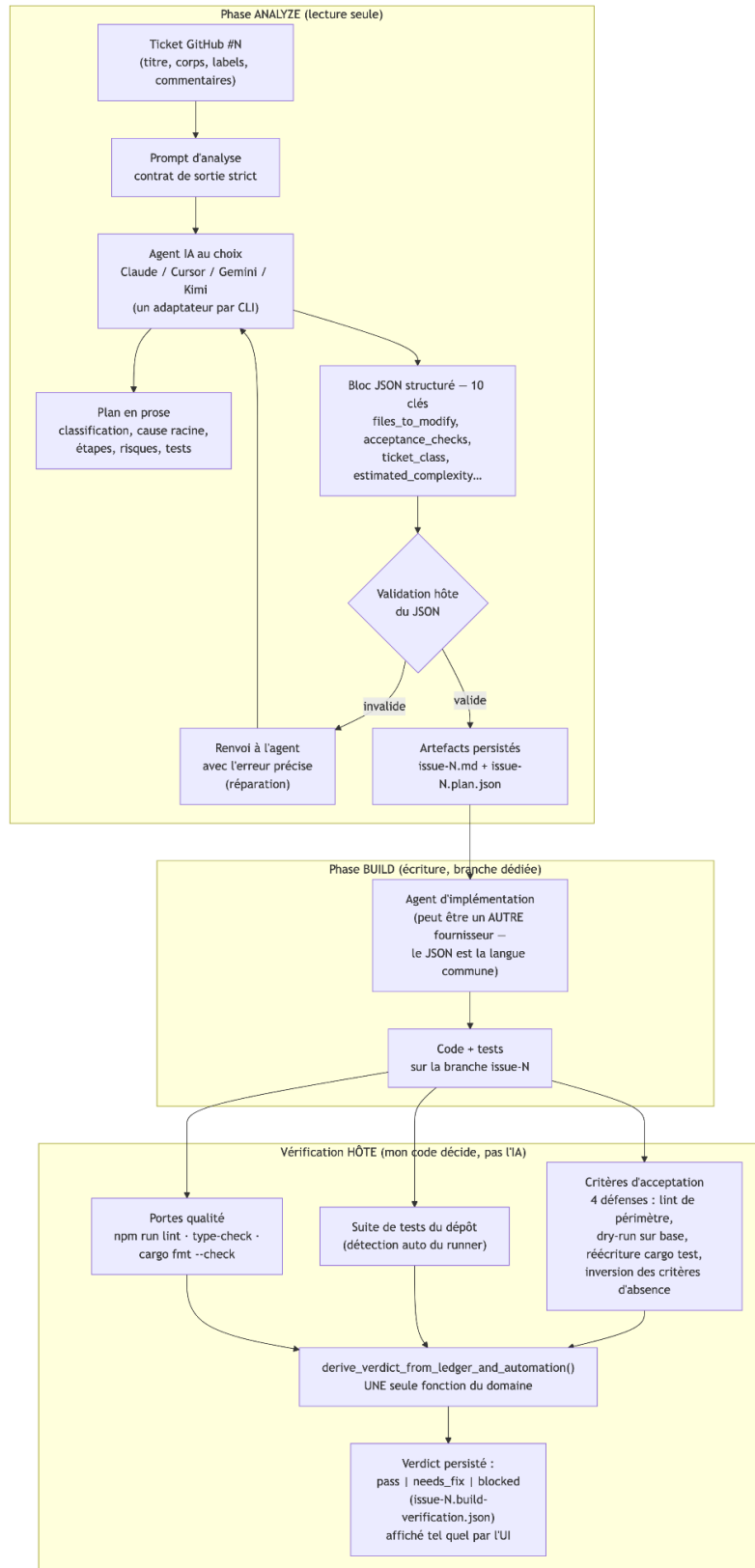


DOSSIER PROFESSIONNEL (DP)



DOSSIER PROFESSIONNEL (DP)

Orchestration des agents IA – du ticket au verdict (kanban-agent)



DOSSIER PROFESSIONNEL ^(DP)

☰

konstantine-garozashvili / kanban-agent

<> Code

🔍 Issues 36

🔗 Pull requests 3

👤 Agents

💬 Discussions

🔗 Actions

📁 Projects

🛡️ Security and quality 2

📊 Insights

⚙️ Settings

🔍 Type / to search

🔍

+

🔄

🔍

📁

👤

< CI

✅ perf(news): memoize thumbnail data URL requests in the frontend #1214

Re-run all jobs

⋮

🏠 Summary

All jobs

📄

🟢 Rust Check & Lint

🟢 Frontend Check & Lint

🟡 Tauri Build Check (macOS)

Run details

🕒 Usage

📄 Workflow file

> Annotations

1 warning

Rust Check & Lint

succeeded 2 days ago in 6m 58s

🔍 Search logs

🔄

⚙️

> 🟢 Set up job

2s

> 🟢 Checkout

2s

> 🟢 Install system dependencies

2s

> 🟢 Install Rust toolchain

1s

> 🟢 Setup Node.js

15s

> 🟢 Stage Tauri bundle resources

7s

> 🟢 Cache Cargo registry

9s

> 🟢 Check formatting

1s

> 🟢 Run Clippy

2m 3s

> 🟢 Run tests

3m 4s

> 🟢 Clean Rust target to save disk

2s

> 🟢 Post Cache Cargo registry

5s

> 🟢 Post Setup Node.js

8s

> 🟢 Post Checkout

1s

> 🟢 Complete job

8s

konstantine-garozashvili

kanban-agent

Q Type to search

<> Code

Issues 36

Pull requests 3

Agents

Discussions

Actions

Projects

Security and quality 2

Insights

Settings

< CI

perf(news): memoize thumbnail data URL requests in the frontend #1214

Re-run all jobs

Summary

All jobs

Rust Check & Lint

Frontend Check & Lint

Tauri Build Check (macOS)

Run details

Usage

Workflow file

Annotations

1 warning

Frontend Check & Lint

succeeded 2 days ago in 2m 35s

Search logs

> Set up job

1s

> Checkout

1s

> Install system dependencies

5s

> Setup Node.js

17s

> Install dependencies

6s

> Type check

17s

> Lint

15s

> Run tests with coverage

1m 28s

> Post Setup Node.js

0s

> Post Checkout

0s

> Complete job

0s

DOSSIER PROFESSIONNEL (DP)

 konstantine-garozashvili

Overview

Repositories 67

Projects

Packages

Stars 1

Q Type to search



Konstantine Garozashvili
konstantine-garozashvili

Full-Stack Developer | Rust & Node.js |
Building AI-powered tools | Alternance @
laplateformeio

Edit profile

11 followers · 7 following

@laplateformeio

Marseille

<https://konstantine.fr>

in/konstantine-garozashvili

Achievements

Organizations



konstantine-garozashvili / README.md

Hi, I'm Konstantine Garozashvili 🙌

Full-Stack Developer · Rust & Node.js · AI-powered tools · Based in Marseille 🇫🇷

PORTFOLIO

KONSTANTINE.FR

LINKEDIN

KONSTANTINE-GAROZASHVILI

EMAIL

GAROZASHVILI25@GMAIL.COM

About Me

I'm a Full-Stack Developer based in Marseille, France, passionate about web development, open source, and building AI-powered applications. Currently in alternance at @laplateformeio, France's leading tech school.

- 🔧 Currently building **kanban-agent** — an AI-powered Kanban tool with Rust + Tauri + React
- 🌱 Deep-diving into Rust, DevOps, Kubernetes and Cloud architecture
- 💬 Ask me about Node.js, React, Rust, Docker, API design
- 🌍 Languages: English (C1) · French (B2) · Russian (C1) · Georgian (C2)
- 📍 Marseille, 13016

Tech Stack

Frontend

Backend

Tools & DevOps

GitHub Stats

DOSSIER PROFESSIONNEL-Version Traitement de texte – Version du 11/09/2017

Page 33